HELM

Product Manual

Command every model. Own the stack.

The enterprise AI governance control plane. Codebase: `management-harness` (Go).

---

1. What Helm Is

Helm is a single control plane between a company's AI tools (Claude Desktop/Code, Cursor, Cline, custom agents) and upstream LLMs (OpenAI, Anthropic, NVIDIA NIM, Nous, and free-tier suppliers). It governs every model call: **who** may call (hashed `hk_` keys + roles), **what it costs** (soft/hard budgets + cost-aware routing, GPU-hour for self-hosted), **what it may say** (a security Guardian, policy engine, PII redaction), and **records it all** (an immutable hash-chained audit log + cryptographically-signed provenance).

Pillar	Components
Govern & secure	auth, guardian (injection/jailbreak/PII), policy engine
Cost & FinOps	budgets, model router, circuit breakers, prompt optimizer, GPU-hour local pricing
Audit & trust	SHA-256 hash-chained audit + `/verify`, Proof of Source / Proof of Human provenance
Productivity	reusable “skills”, Draft-with-AI wizard, optimizer telemetry

2. Quick Start

```
go build ./...                    # or: make build
# two services (share Redis for cross-process state):
MH_REDIS_ENABLED=true ./bin/management-harness-proxy &
MH_MANAGER_ADMIN_TOKEN=... MH_MANAGER_JWT_SECRET=... ./bin/management-harness-manager &
```

- **Proxy** :8080 — OpenAI-compatible `/v1/chat/completions`, `/stream`, `/v1/messages`
- **Manager** :8090 — `/api/v1/...` + embedded dashboard at `/`
- Readiness: `/healthz`, `/readyz`, `/livez`, `/metrics`

Open `http://localhost:8090/`, sign in with the manager admin token, create an operator + mint an `hk_` key, and call the proxy with `Authorization: Bearer hk_...`.

3. Architecture

```
AI Tool ──► PROXY ─ auth ─ guardian(ORIGINAL) ─ [skill expand] ─ guardian(re-scan)
               ─ optimize ─ budget(EstimateCost) ─ router/selectProvider ─► LLM
                 ├─ Metrics (Prometheus /metrics)
                 ├─ Audit (hash-chained, immutable)
                 └─ Manager API + Dashboard (:8090)
```

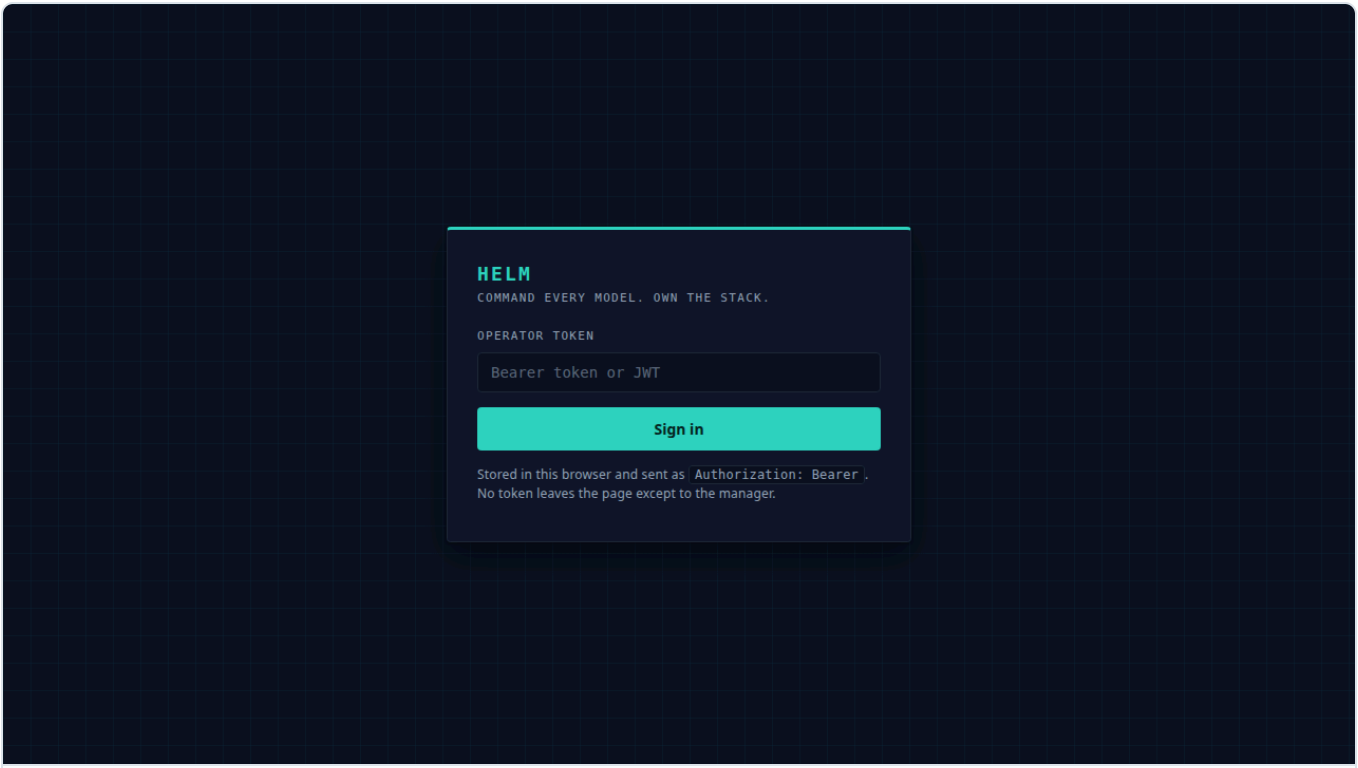
Guardian always scans the **original** prompt; a used **skill** is re-scanned; optimization runs after the guardian and before budget/routing so the charge matches the tokens actually sent.

4. Concepts

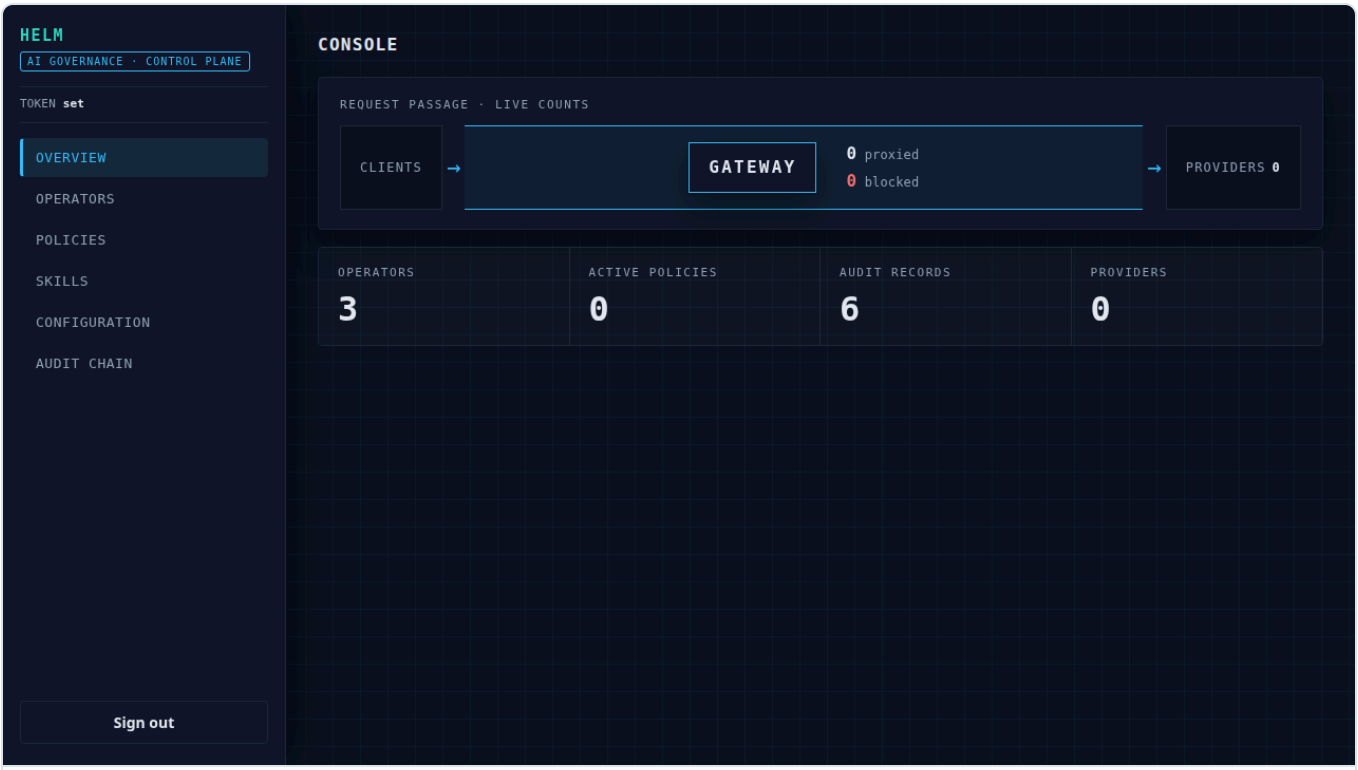
Concept	Description
Identity	User/team/org + roles resolved from the key; drives budgets + authorization.
API keys	`hk_`-prefixed, SHA-256 hashed; minted once via Manager.
Budget	Per-identity soft (warn) / hard (block) USD limits; Redis-backed.
Guardian	Blocks injection/jailbreak/tool-escalation; redacts 18 PII patterns.
Policy	Priority-ordered, hot-reloadable rules.
Router / breaker	Cost/health routing + flapping-provider protection.
Audit	Immutable SHA-256 hash chain; `/verify` proves integrity.
Skill	Reusable prompt contract (system + `{{slot}}` template + params + permissions).

Optimizer	Lossless or LLM-based compression, skill-scoped.
Provenance	Ed25519 Proof of Source / Proof of Human on ingested data.

5. Dashboard Tour (dark “operators at night” theme)

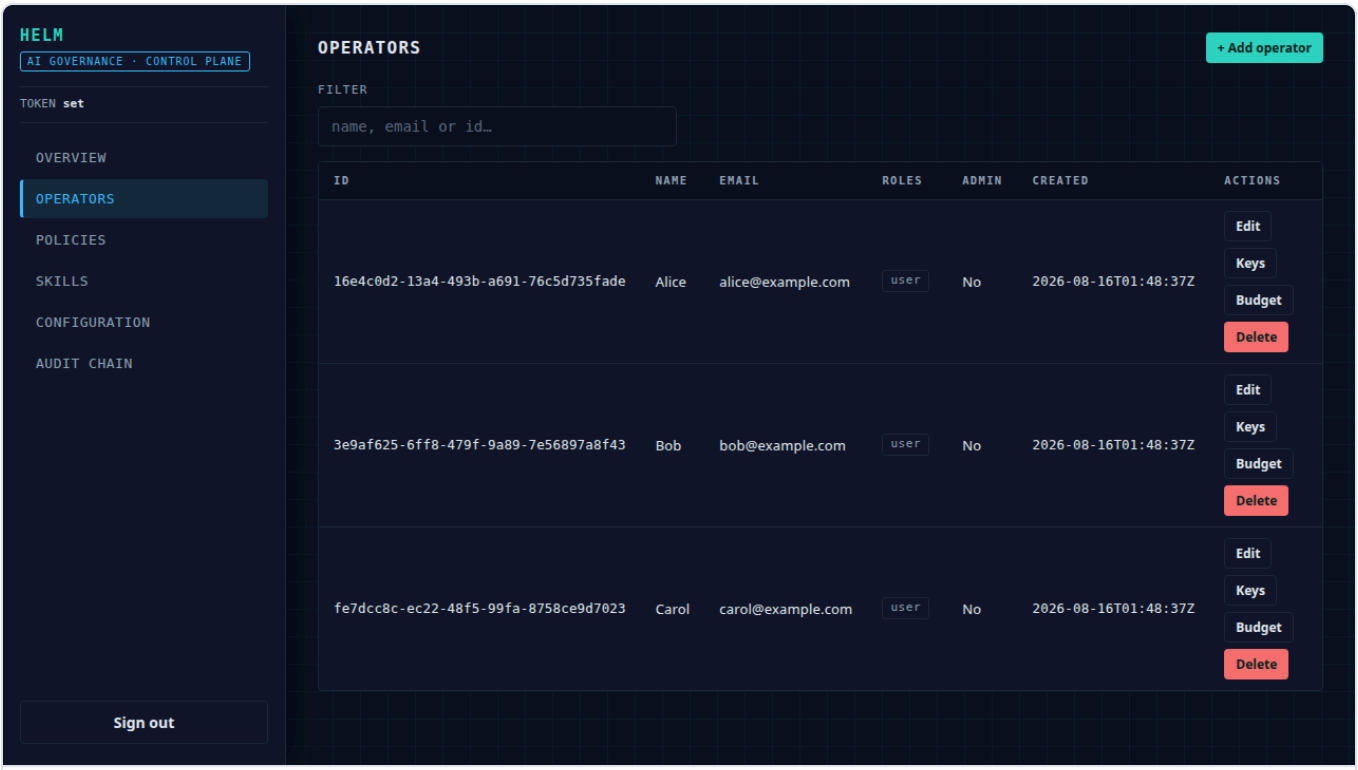


Login

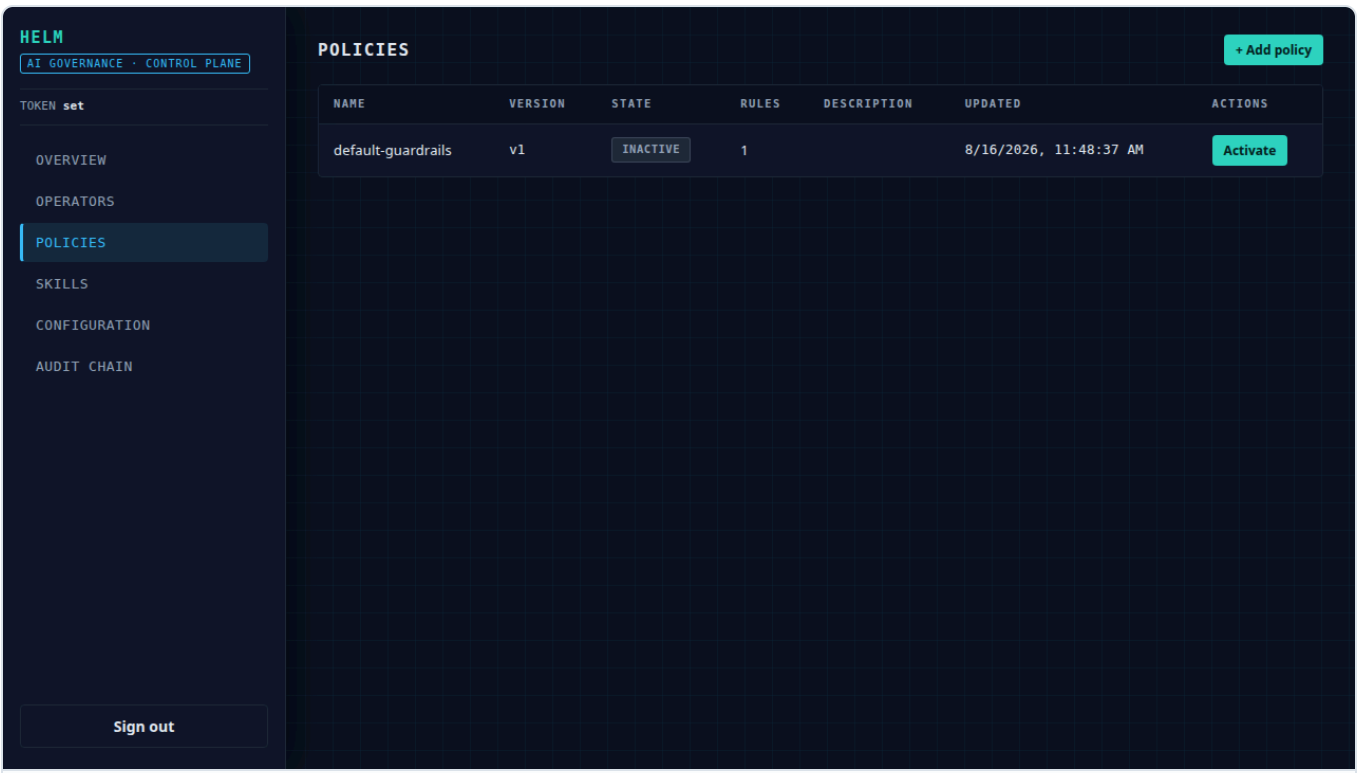


Overview

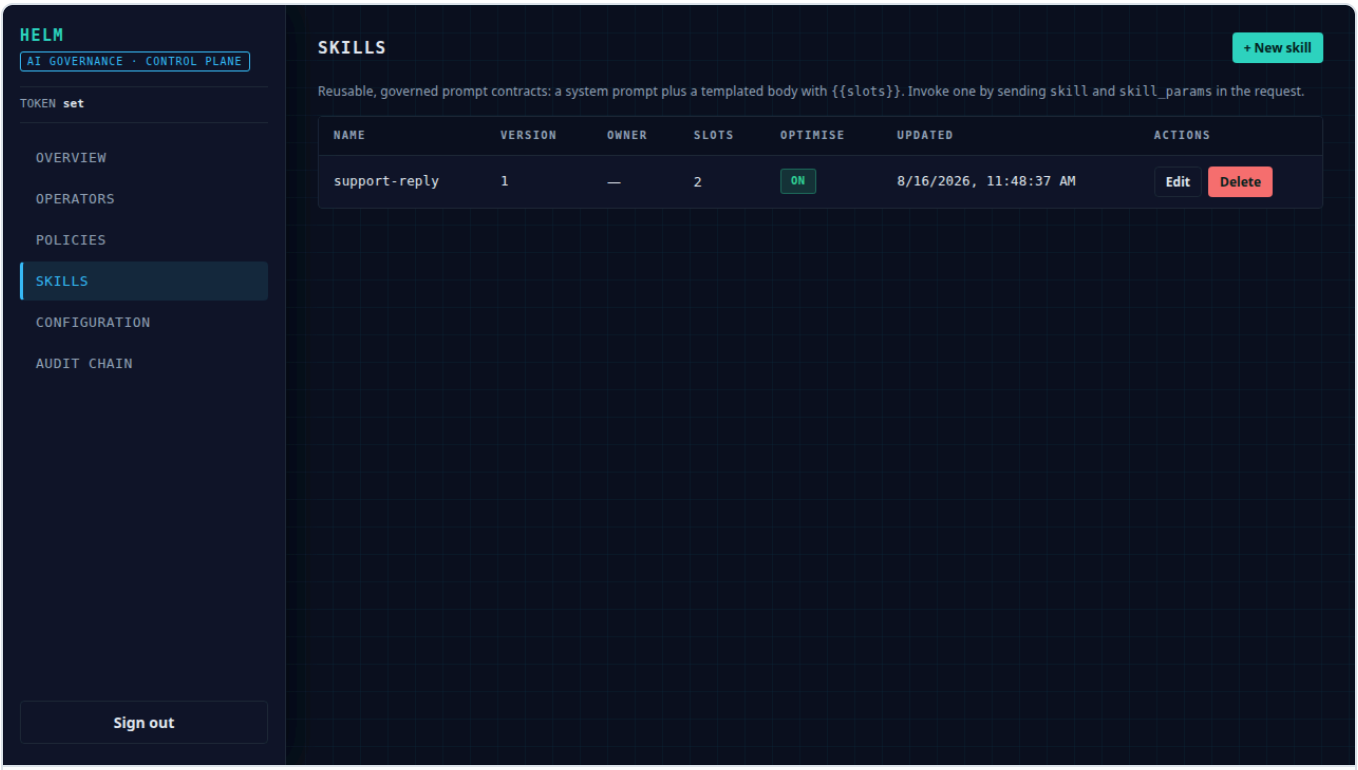
Page	What it does
Overview	Live counts: requests, threats blocked, operators, policies, audit records, providers.
Operators	Create/edit/delete users; mint + revoke `hk_` keys; set per-user budgets.
Cost	Per-operator budgets + spend, total spend, optimizer tokens saved, users over budget, inline budget editing.
Policies	Create/activate hot-reloadable policies.
Skills	Library of prompt contracts; create/edit + Draft with AI; delete.
Configuration	Edit providers (NVIDIA, Nous, Kilo, OpenCode, local), guardian, proxy, breaker, pricing, optimizer, secrets, Redis.
Audit Chain	Hash-linked immutable log with live chain-integrity verification.



Operators



Policies


HELM
AI GOVERNANCE · CONTROL PLANE
TOKEN set
OVERVIEW
OPERATORS
POLICIES
SKILLS
CONFIGURATION
AUDIT CHAIN
Sign out
SKILLS
+ New skill
Reusable, governed prompt contracts: a system prompt plus a templated body with {{slots}}. Invoke one by sending skill and skill_params in the request.
NAME
VERSION
OWNER
SLOTS
OPTIMISE
UPDATED
ACTIONS
support-reply
1
—
2
ON
8/16/2026, 11:48:37 AM
Edit
Delete

Skills

HELM

AI GOVERNANCE · CONTROL PLANE

TOKEN **set**

- OVERVIEW
- OPERATORS
- POLICIES
- SKILLS
- CONFIGURATION
- AUDIT CHAIN

Sign out

CONFIGURATION

Save changes

Providers and their API keys reload **live**. Other settings persist and apply on restart. Secrets are masked — leave a secret field blank to keep the existing value.

PROVIDERS

+ Add provider

GUARDIAN

- [x] Enabled
- [x] Block prompt injection
- [x] Block jailbreak
- [x] Redact PII

PROXY

DEFAULT PROVIDER

openai

- [x] Require authentication

MAX REQUEST BYTES

4194304

CIRCUIT BREAKER

MAX FAILURES

5

COOLDOWN (E.G. 10S)

10s

PRICING & METRICS

DEFAULT RATE (USD / 1K TOKENS)

3

- [x] Metrics enabled

OPTIMISER / NVIDIA

- [ ] Enable LLM prompt optimisation + skill wizard

PROVIDER (MUST MATCH A NAME ABOVE, E.G. NVIDIA)

nvidia

MODEL

nvidia/nemotron-3.5-lightning-30b-a3b

- [ ] Shadow mode (measure savings, apply nothing)

SECRETS & REDIS

ADMIN TOKEN (BLANK TO KEEP)

•••• set

JWT SECRET (BLANK TO KEEP)

•••• set

- [x] Redis enabled

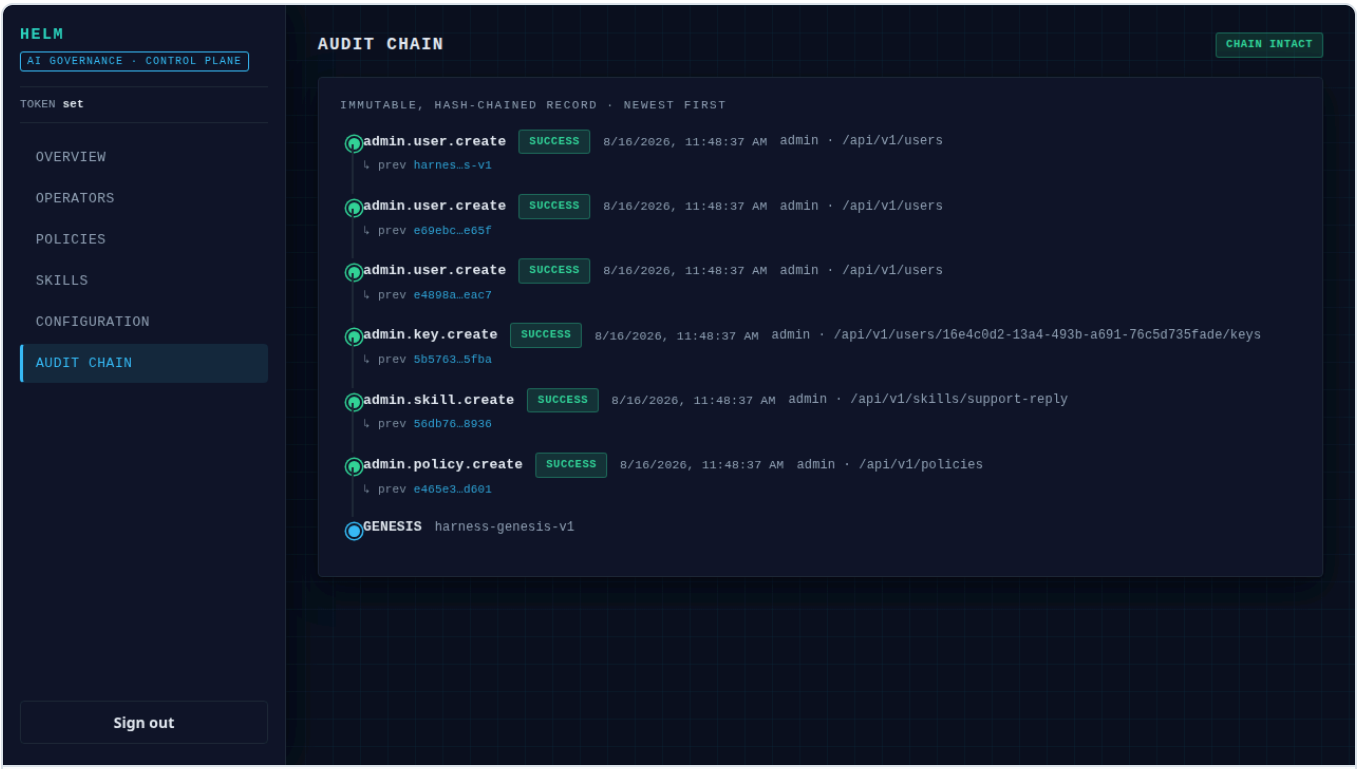
REDIS ADDRESS

localhost:6379

REDIS PASSWORD (BLANK TO KEEP)

not set

Configuration



Audit Chain

6. Configuration

Static = `configs/config.yaml` + `MH_*` env (`.env.example`); **runtime** = editable on the dashboard, persisted to `configs/runtime.json` . Providers/API keys reload live; other settings apply on restart.

NVIDIA & free/open-source suppliers

All are OpenAI-compatible — add your key on the dashboard (Configuration → Providers → the provider → tick Enabled → Save).

Provider	base_url	Example models
nvidia	`https://integrate.api.nvidia.com/v1`	nvidia/nemotron-3.5-lightning-30b-a3b
nous	`https://inference-api.nousresearch.com/v1`	Hermes-4-405B, Hermes-4-70B

kilo	`https://api.kilo.ai/api/gateway`	openrouter/free, stepfun/step-3.7-flash:free
opencode	`https://opencode.ai/zen/v1`	opencode/hy3-free, opencode/deepseek-v4-flash-free
local	your vLLM/llama.cpp	any self-hosted model (GPU-hour costed)

> OpenCode Zen serves GPT/Grok over a Responses/messages API — use its /chat/completions-compatible free models here. Nous “free” is a rate-limit tier (quota unverified). Kilo base_url per docs; if your engine expects a trailing `/v1`, append it.

Shipped disabling by default; enable per-org.

7. Local inference (self-hosted) & GPU-hour accounting

```
providers:
  local:
    type: local
    base_url: "http://vllm:8000/v1"
    enabled: true
    local:
      gpu_hour_cost: 2.5
      gpu_count: 1
      declared_throughput_tps: 90
      safety_factor: 1.25
      min_gpu_seconds_per_request: 3
```

A shared H100-70B lands ~$1–3/1M tokens (vs ~$1k+ hosted); the per-request GPU-second floor prevents tiny requests from evading the budget. Budget pre-check and post-usage both use this model, so the **Cost** page shows real GPU cost.

8. Skills

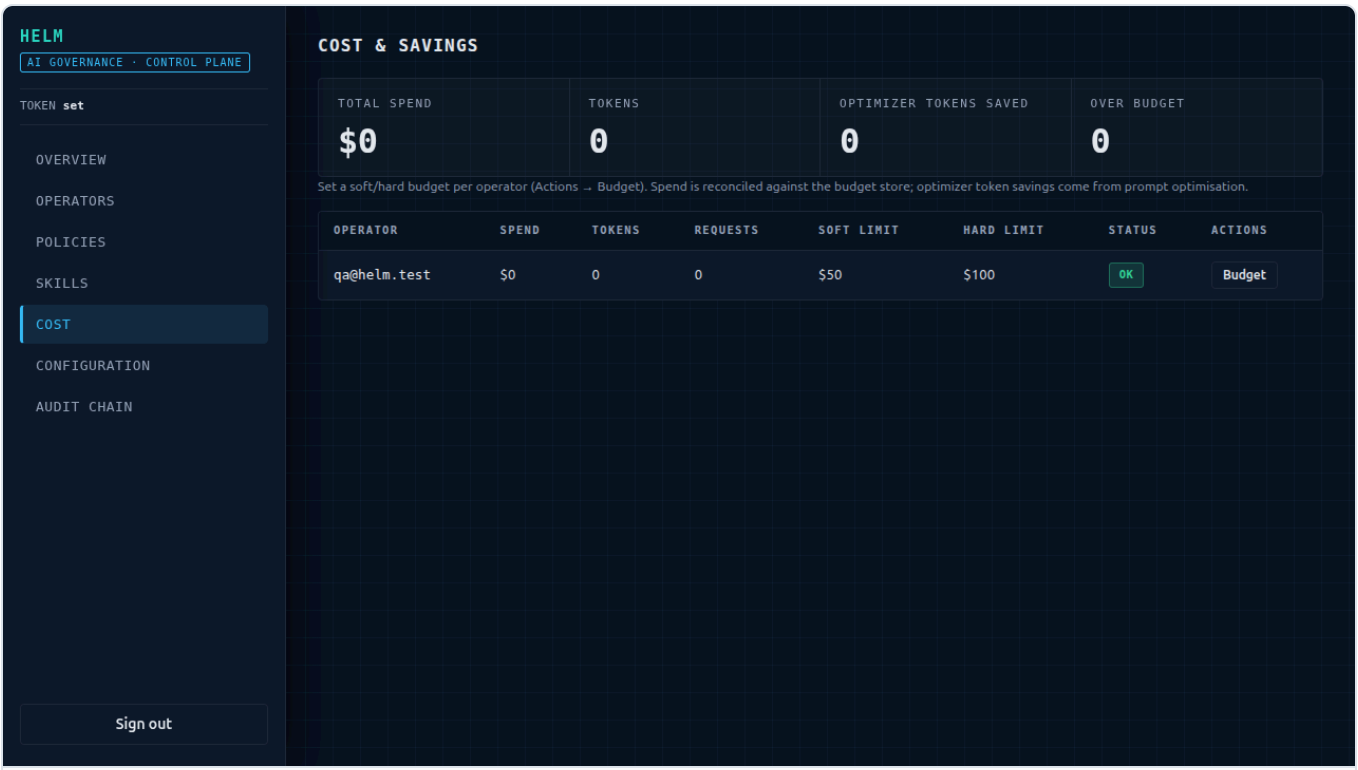
A **Skill** = `system_prompt` + `template` (with `{{slot}}`) + `parameters` + `permissions`. Create via the dashboard or **Draft with AI** (the configured LLM generates a full skill from a description). Invoke by adding fields to the request:

```
{ "model":"gpt-4o", "skill":"support-reply",
  "skill_params": { "message":"My order hasn't arrived", "tone":"professional" } }
```

Unknown skill → `404`; unauthorized → `403`; invalid params → `400`. Optimization opt-in per skill via `optimize:true`.

9. Prompt optimization & cost

Lossless (default): collapses redundant whitespace. **LLM-based (via `optimizer`):** a provider model compresses the prompt behind a semantic-preservation gate. Savings appear on the **Cost** page and as `harness_optimizer_*` metrics. The **Cost** page aggregates per-operator budgets, total spend, and optimizer tokens saved.



Cost & Savings

10. Provenance — Proof of Source / Proof of Human

Sign data entering critical pipelines with an attested **source** (`human` / `model` / `unknown`) and verify it wasn't synthetically altered or model-poisoned. Ed25519-signed attestations bind the payload's SHA-256 hash to a signer + source + timestamp.

- **Sign:** `POST /api/v1/provenance/sign {source, payload_base64}` → a signed `record` (admin).
- **Verify:** `POST /api/v1/provenance/verify {record, payload_base64}` → `{"verdict":{...},"proof_of_human":true|false}`. Payload change or source re-attribution fails.
- **Middleware:** `provenance.Middleware(pub, requireValid)` on ingestion routes verifies an `X-Helm-Provenance` header and fail-closes (403) on tampered data, or exposes `Verdict.HumanProof()`.
- **Key:** `GET /api/v1/provenance` returns the verifier public key; set `MH_PROVENANCE_PRIVATE_KEY` (base64) for persistence across restarts/air-gap.

Use case: gate RAG/enterprise ingestion on `proof_of_human`, or quarantine anything signed `model` (synthetic).

11. Security & compliance

- Keys hashed at rest; secrets never leave as plaintext (masked in dashboard).
- Guardian blocks prompt injection, jailbreaks, tool escalation; redacts PII.
- Immutable audit chain: `GET /api/v1/audit/verify` proves integrity; `/audit/export` for SIEM.
- Provable provenance (above) for data-integrity and anti-poisoning.

12. API Reference (Manager, `/api/v1`)

Method	Path	Description
GET/POST	/users, /users/{id}	Users CRUD
GET/POST	/users/{id}/keys, /revoke	Mint/revoke keys
GET/PUT	/users/{id}/budget, /budget/usage	Get/set budgets
GET/POST	/policies, /policies/{id}/activate	Policies
GET/POST	/skills, /skills/draft, /skills/{name}	Skills CRUD + Draft with AI
GET/PUT	/config	Read (masked) / write runtime config

GET	/cost	Aggregate cost, savings, per-operator budgets
GET/POST	/provenance, /provenance/sign, /provenance/verify	Proof of Source / Proof of Human
GET	/overview, /audit/events, /audit/verify	Counts / audit

13. Operations & editions

- **Metrics:** Prometheus on `/metrics`; `prometheus.yml` in `deployments/`.
- **Readiness:** `/readyz` verifies Redis + a healthy provider.
- **Docker:** `docker build -t management-harness .`; run proxy (default) or `docker run ... /app/manager`.

Editions (see `PRICING_STRATEGY.md`): Community (self-host, $0) → Pro/Scale → Enterprise (SSO/RBAC/air-gap/SIEM). Core governance ships in every edition.

14. Troubleshooting

Symptom	Fix
503 “no provider available”	Enable a provider in Configuration → Providers.
401 “invalid API key”	Enable `MH_REDIS_ENABLED=true` so manager-minted keys are shared with the proxy.
“insecure default” panic	Set real `MH_MANAGER_ADMIN_TOKEN` / `MH_MANAGER_JWT_SECRET`.
Settings saved but proxy unchanged	Providers/keys reload live; other settings apply on restart.
Provenance fails after restart	Set a persistent `MH_PROVENANCE_PRIVATE_KEY` (else key is ephemeral).

> Brand: **Helm** — “Command every model. Own the stack.” Full strategy: `PRICING_STRATEGY.md` , `docs/BRAND-AND-PRICING-recommendation.md` .